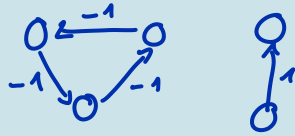


Suppose a directed graph has a negative cycle. Then the distance between any two vertices is $-\infty$.

☐ True

☒ False



bellmanford3

11 min

Suppose that during the execution of Dijkstra's algorithm to find a path from s to t in a directed graph with non-negative edge weights, the array d of distance bounds is as follows:

v	s	v_1	v_2	v_3	v_4	t
$d(v)$	0	6	4	3	8	5

Which of the following statements must be true **after Dijkstra's algorithm terminates**?

☐ a. $d(t) = 5$ könnte sich noch verbessern

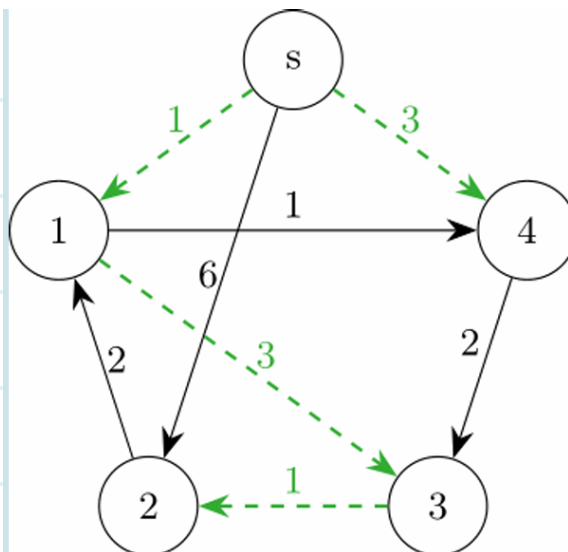
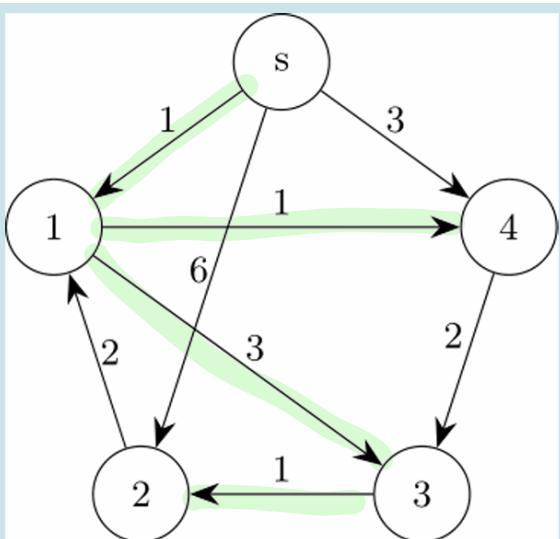
☒ b. $d(v_3) = 3$ kann sich nicht mehr verbessern weil alle Kantengewichte ≥ 0 sind

☐ c. $d(v_4) < 8$ muss sich nicht zwingend noch verbessern

On directed graphs, Dijkstra's algorithm can be implemented in time $O((n + m) \log(n))$ (in the worst case) where n is the number of vertices and m is the number of edges.

☒ True

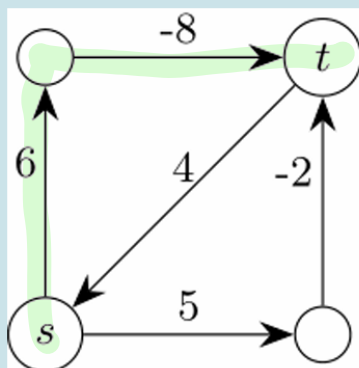
☐ False



Consider the graph in the first picture. Is the tree, denoted by green dashed edges on the second picture, a shortest path tree starting from s in the graph?

☐ True

☒ False



Suppose we apply the Bellman-Ford algorithm to the weighted, directed graph above to find a shortest path from s to t . True or false: we will have the correct distance to t after the *improve bounds* procedure (as in lecture) has been executed 3 times.

☒ a. True

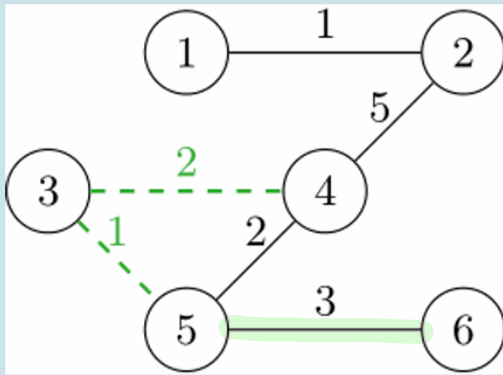
☐ b. False

☐ c. This depends on the order in which the vertices are processed in the *improve bounds* procedure

Consider a weighted directed graph G with potentially negative edge weights but no negative cycles. What is the minimum number of times the improve bounds procedure in the Bellman-Ford algorithm needs to be executed in order to guarantee that the correct distances have been computed (in the worst case; where n is the number of vertices)?

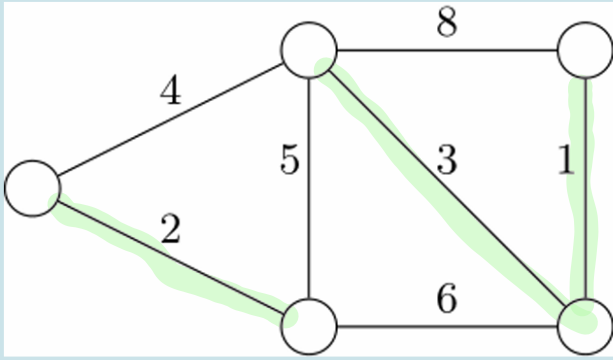
- ☐ a. 1
- ☐ b. 2
- ☒ c. $n - 1$
- ☐ d. n
- ☐ e. $n + 1$

Ein Pfad kann höchstens n Knoten und $n-1$ Kanten haben



In the weighted graph above, the edges of an (incomplete) minimum spanning tree constructed during the execution of Prim's Algorithm are marked (dashed and green). Which edge will be added to the tree next?

- ☐ a. {1, 2}
- ☐ b. {2, 4}
- ☐ c. {4, 5}
- ☒ d. {5, 6}



Boruvka's algorithm finds a minimum spanning tree by adding groups of edges in steps. How many edges does Boruvka's algorithm add in the first step when applied to the weighted graph above?

Answer:

3

Let G be an undirected, weighted, connected graph. Suppose that there is a unique edge e with weight 1 and all other edges have weight strictly greater than 1.

Then any MST of G contains the edge e .

☒ True

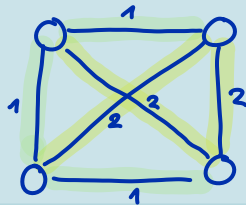
☐ False

Boruvka wählt e im ersten Schritt

Let $G = (V, E)$ be an undirected, connected graph with positive edge weights. Suppose that G has a unique minimum spanning tree $T \subseteq E$. Consider any *spanning* subset $S \subseteq E$ of G . Recall that a subset $S \subseteq E$ is called *spanning* if the graph (V, S) is a connected graph. Then S must share an edge with T .

☐ True

☒ False



Bellman-Ford

Bedingungen: negative Kantengewichte sind erlaubt, aber es darf keinen negativen Zyklus geben

$S_{\leq \ell} = \{v \in V \mid \exists \text{ ein kürzester (= günstigster) Weg von } s \text{ zu } v \text{ mit } \leq \ell \text{ Kanten}\}$

ℓ -genaue Schranke: alle kürzesten (= günstigsten) Wege mit $\leq \ell$ Kanten sind bereits gefunden

$$\text{d.h. } d[v] = d(s, v) \quad \forall v \in S_{\leq \ell}$$

$$d[v] \geq d(s, v) \quad \forall v \notin S_{\leq \ell}$$

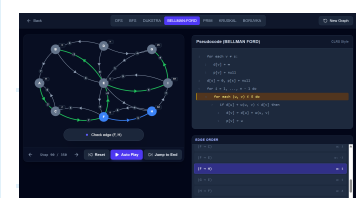
BELLMAN-FORD($G = (V, E), s$) in $O(n \cdot m)$

1 for each $v \in V \setminus \{s\}$ do	} 0-genaue Schranken	▷ Initialisiere für alle Knoten die
2 $d[v] \leftarrow \infty$; $p[v] \leftarrow \text{null}$		▷ Distanz zu s sowie Vorgänger
3 $d[s] \leftarrow 0$; $p[s] \leftarrow \text{null}$		▷ Initialisierung des Startknotens
4 for $i \leftarrow 1, 2, \dots, V - 1$ do		▷ Wiederhole $ V - 1$ Mal
5 for each $(u, v) \in E$ do		▷ Iteriere über alle Kanten (u, v)
6 if $d[v] > d[u] + w((u, v))$ then		▷ Relaxiere Kante (u, v)
7 $d[v] \leftarrow d[u] + w((u, v))$		▷ Berechne obere Schranke
8 $p[v] \leftarrow u$		▷ Speichere u als Vorgänger von v
9 for each $(u, v) \in E$ do		▷ Prüfe, ob eine weitere Kante
10 if $d[u] + w((u, v)) < d[v]$ then		▷ relaxiert werden kann
11 Melde Kreis mit negativem Gewicht		

Invariante: i -genaue Schranken nach der i -ten Iteration

Achtung: Reihenfolge der Kanten ist nicht vorgegeben ($\rightarrow$ Quizfrage)

$\exists$ neg Zyklus falls sich nach der n -ten Iteration eine Distanz ändert



MST

Sei $G = (V, E)$ ein zusammenhängender Graph

Baum: ungerichteter, zusammenhängender, kreisfreier Graph

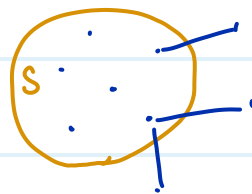
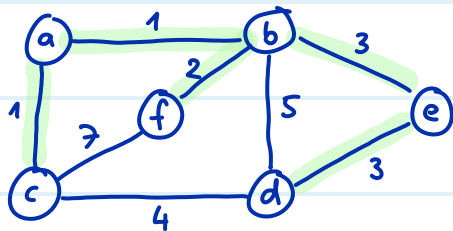
Aufspannende Kanten $A \subseteq E$ falls $G(V, A)$ zusammenhängend ist

Spannbaum $T \subseteq E$ falls T aufspannend ist und keinen Kreis hat

Minimaler Spannbaum $MST \subseteq E$ falls MST ein Spannbaum mit minimalem Gewicht ist

Sichere Kante $e \in E$ falls e in jedem MST vorkommt

- Kante mit kleinstem Gewicht an jeden Knoten ist sicher
- Kante mit kleinstem Gewicht an jeden Schnitt ist sicher



Ist $A = \{\{a,b\}, \{a,c\}, \{f,b\}, \{e,d\}\}$ aufspannend? **nein**

Welche Kanten könnte man zu A hinzufügen, sodass A aufspannend ist? $\{b,e\}, \{b,d\}, \{c,d\}$ und weitere

Welche Kanten könnte man zu A hinzufügen, sodass A ein Spannbaum ist? $\{b,e\}, \{b,d\}, \{c,d\}$

Welche Kanten könnte man zu A hinzufügen, sodass A ein minimaler Spannbaum ist? $\{b,e\}$

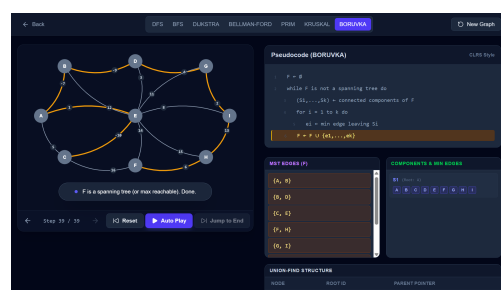
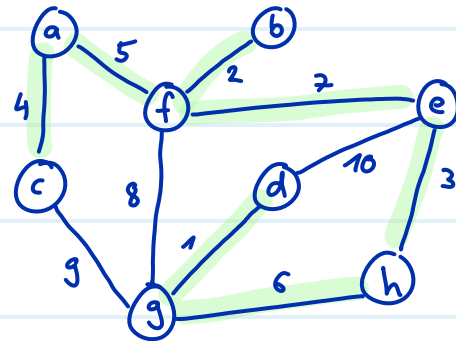
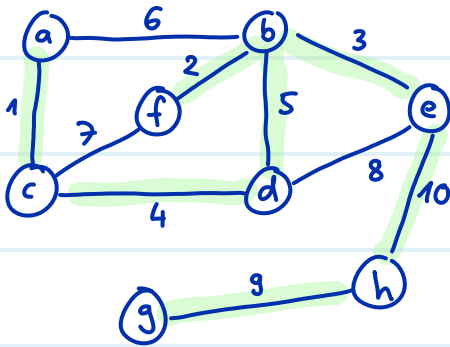
Welches ist die Kante mit kleinstem Gewicht an Schnitt $S = \{a,b,d\}$ $\{a,c\}$

Wie viele sichere Kanten gibt es? **5**

Algorithm 1 BORUVKA(G) in $O((n+m) \cdot \log n)$

- 1: $F \leftarrow \emptyset$
 - 2: **while** F is not a spanning tree **do**
 - 3: $(S_1, \dots, S_k) \leftarrow$ connected components of F
 - 4: $(e_1, \dots, e_k) \leftarrow$ minimum edges leaving $S_1, \dots, S_k$
 - 5: $F \leftarrow F \cup \{e_1, \dots, e_k\}$
-

Bedingung: paarweise verschiedene Kantengewichte

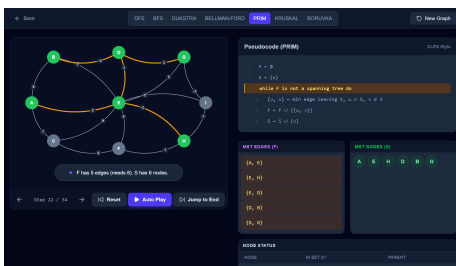
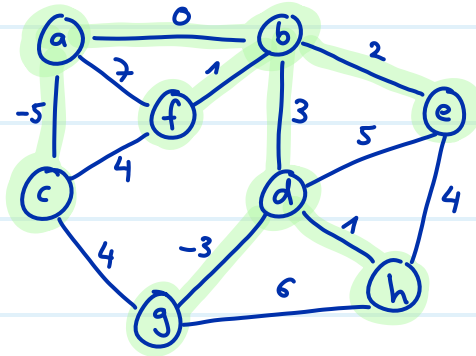
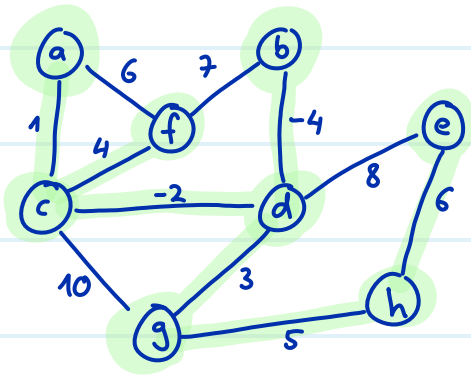


Algorithm 1 PRIM(G, s) in $O((n+m) \log n)$

- 1: $F \leftarrow \emptyset$
- 2: $S \leftarrow \{s\}$
- 3: **while** F is not a spanning tree **do**
- 4: $\{u, v\} \leftarrow$ minimale Kante an S , mit $u \in S, v \notin S$
- 5: $F \leftarrow F \cup \{\{u, v\}\}$
- 6: $S \leftarrow S \cup \{v\}$

• mehrere Kanten mit gleichem Gewicht erlaubt

Bsp Startknoten a



- **Dijkstra:** Read the **blue lines** and ignore the orange ones.
- **Prim:** Read the **orange lines** and ignore the blue ones.

Algorithm 1 DIJKSTRA / PRIM ($G = (V, E), s$)

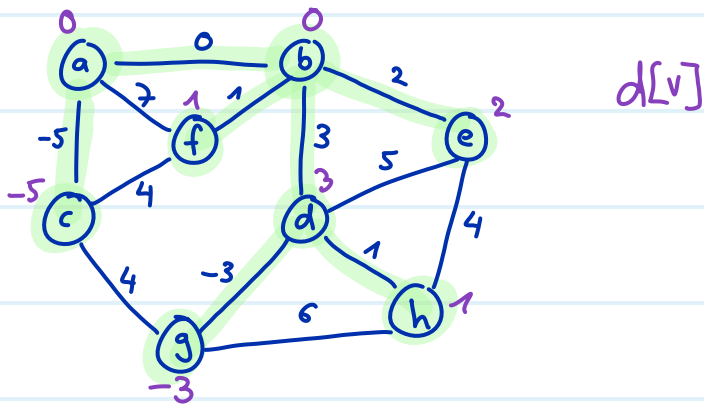
 in $O((n + m) \cdot \log n)$

```

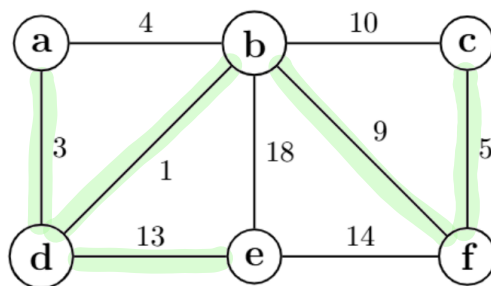
1: for each  $v \in V \setminus \{s\}$  do
2:    $d[v] \leftarrow \infty; p[v] \leftarrow \text{null}$ 
3:  $d[s] \leftarrow 0; p[s] \leftarrow \text{null}$ 
4:  $Q \leftarrow \emptyset$ 
5: INSERT( $s, 0, Q$ )
6: while  $Q \neq \emptyset$  do
7:    $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
8:   for each  $\{u, v\} \in E$  do
9:     if  $p[v] = \text{null}$  then
10:       $d[v] \leftarrow d[u] + w(\{u, v\})$ 
11:       $d[v] \leftarrow w(\{u, v\})$ 
12:       $p[v] \leftarrow u$ 
13:      ENQUEUE( $v, d[v], Q$ )
14:     else if  $d[u] + w(\{u, v\}) < d[v]$   $w(\{u, v\}) < d[v]$  then
15:       $d[v] \leftarrow d[u] + w(\{u, v\})$ 
16:       $d[v] \leftarrow w(\{u, v\})$ 
17:       $p[v] \leftarrow u$ 
18:      DECREASE-KEY( $v, d[v], Q$ )

```

▷ Initialisiere für alle Knoten
 ▷ Distanz/Kosten sowie Vorgänger
 ▷ Init. Startknoten (Dijkstra) / Wurzel (Prim)
 ▷ Leere Prioritätswarteschlange Q
 ▷ Füge s zu Q hinzu
 ▷ Wähle Knoten mit min. d -Wert
 ▷ Inspiziere Nachfolger
 ▷ v wurde noch nicht entdeckt
 ▷ **Dijkstra: Berechne Distanz**
 ▷ **Prim: Berechne Kosten**
 ▷ Speichere u als Vorgänger
 ▷ Füge v zu Q hinzu
 ▷ **Dijkstra: Update Distanz**
 ▷ **Prim: Update Kosten**
 ▷ Vorgänger aktualisieren
 ▷ Priorität anpassen



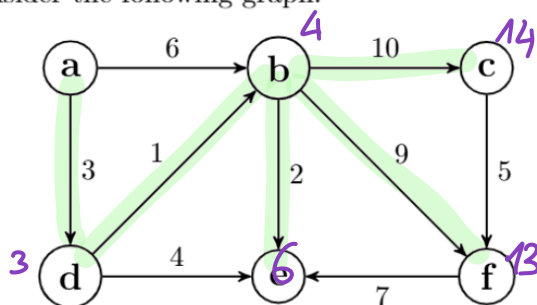
/ 2 P

e) *Minimum Spanning Tree*: Consider the following graph:

- i) Highlight the edges that are part of the minimum spanning tree. (Either in the picture above, or you can recreate the graph below).
- ii) Write out all positive integers x such that if we replace the weight 1 of edge $\{b, d\}$ in the above graph with x , then edge $\{b, d\}$ would be in at least one minimum spanning tree of the resulting graph.

(1), 2, 3, 4

/ 2 P

f) *Shortest Path Tree*: Consider the following graph:

- i) Highlight the edges that are part of the shortest-path tree rooted at vertex a (i.e., the output of Dijkstra's algorithm if we were to start from vertex a).
- ii) Does the above graph have a topological ordering? If yes, write down one topological ordering. If no, give an argument.

a, d, b, c, f, e

Theory Task T4.

/ 14 P

The city center of Amsterdam is known for its many tourists and many bridges. There are $n \geq 4$ tourist attractions in Amsterdam labeled $\{1, 2, \dots, n\}$. For some pairs $i, j \in \{1, 2, \dots, n\}$, the attractions i, j are connected by a two-way bridge, represented by $\{i, j\}$. The set of all bridges is denoted with $B = \{\{i_1, j_1\}, \{i_2, j_2\}, \dots, \{i_m, j_m\}\}$. You may assume that $m \geq n$. Using the bridges, any attraction i can be reached from any other attraction j .

Remark: When asked to describe an algorithm, you need to address the following aspects:

- 1) the graph algorithm that you use as a subroutine in your solution;
- 2) the construction of the graph that you run this algorithm on;
- 3) the correctness of your solution, with a short justification (i.e., how and why does running the chosen graph algorithm on the graph you constructed solve the original problem);
- 4) the total running time of your solution, with a short justification.

You can directly use the algorithms covered in the lecture material, and you can directly use their running time bounds without proof.

Remark: It is not needed to solve part a) to attempt parts b) and c). It is not needed to solve any earlier parts to attempt part d).

/ 3 P

- a) To accommodate for boats, the city government wants to remove some of the bridges. Each bridge $b_k = \{i_k, j_k\}$ has a length $L_k \in \mathbb{N}$. Describe an algorithm which outputs a subset $D \subseteq B$ of bridges, of largest possible total length $L(D) := \sum_{k: b_k \in D} L_k$, which can be removed while still making sure that tourists can travel between any pair of attractions $i, j \in \{1, 2, \dots, n\}$ (without swimming). The running time of your algorithm should be $O(m \log m)$.

1) Prim

2) We use an undirected Graph $G = (V, E)$

where $V = \{1, \dots, n\}$ are the attractions

and $E = B$ are the bridges

and each edge $b_k \in B$ has weight L_k

Notice that G is connected

3) Prim gives us a minimum spanning tree $M \subseteq B$.

We return $O := B \setminus M$

Justification: We only remove edges that are not part of the MST and therefore the remaining graph $G' = (V, M)$ is still connected.

Assume that $\bar{D}$ is a better solution, i.e. $L(\bar{D}) > L(D)$ and $\bar{G} = (V, B \setminus \bar{D})$ is connected. Then $L(B \setminus \bar{D}) < L(M)$ which means that M was not a minimal spanning tree. Contradiction.

Therefore our solution is optimal.

4) $m \geq n$ is given

We can construct the graph in $O(n+m) \leq O(m)$

Prim runs in $O((n+m) \cdot \log n) \leq O(m \log m)$

Thus in total we have $O(m \log m)$

It turns out the bridges are historical buildings, and so they cannot be removed. Instead, the government decides to only open each bridge during some parts of the day. They divide a day into $N \geq 2$ units of time, labeled $\{0, 1, \dots, N-1\}$, and give each bridge $b_k = \{i_k, j_k\}$ *opening times* $O_k \subseteq \{0, 1, \dots, N-1\}$. Each bridge also has a *crossing time* $C_k \in \mathbb{N}$, which represents the number of time-units required to cross it. **Tourists are only allowed to start crossing a bridge when it is open** (but it is not a problem if it closes while they are crossing it). Note that a tourist's travel is allowed to span multiple days. For example, they might start to cross a bridge b_k with crossing time $C_k = 4$ at time $T = N-2$, and finish the next day at time $T' = 2$ (as long as $N-2 \in O_k$).

/ 4 P

- b) We are given two attractions $s, t \in \{1, 2, \dots, n\}$, and a starting time $T_{\text{start}} \in \{0, 1, \dots, N-1\}$. Describe an algorithm which **decides if it is possible for tourists to travel from s to t , starting at time T_{start} , while respecting the opening times of the bridges, but **without having to wait** at any of the attractions.** The running time of your algorithm should be $O(m \cdot N)$.

Hint: Use a directed graph whose vertex set consists of N copies of the attractions $\{1, 2, \dots, n\}$, labeled $\{(i, T) : i \in \{1, 2, \dots, n\}, T \in \{0, 1, \dots, N-1\}\}$. When should there be an edge from (i, T) to (i', T') ?

1) DFS or BFS

2) Let $G = (V, E)$ be a directed graph where

$$V = \{(i, T) \mid i \in \{1 \dots n\} \text{ and } T \in \{0, \dots, N-1\}\} \text{ and}$$

$$E = \left\{ ((i, T), (j, S)) \mid b_k = \{i, j\} \in B \text{ and } T \in O_k \text{ and } T + C_k \equiv_N S \right\}$$

The vertex set contains N copies of the attraction set, one copy for every $T \in \{0 \dots N-1\}$.

We add a directed edge from (i, T) to (j, S) if there is a bridge $b_k = \{i, j\} \in B$ that is open at time T , i.e. $T \in O_k$, and such that $T + C_k \equiv_N S$

3) We run DFS starting at $(s, T_{\text{start}}) \in V$

We return "possible" if we visit a vertex $(t, T) \in V$ for any $T \in \{0 \dots N-1\}$ during DFS. Otherwise we return "impossible"

Justification: directed walks in G correspond to all legal walks of tourists in Amsterdam (without waiting) ..

4) $|V| = n \cdot N$ and $|E| \leq m \cdot N$

We can construct the graph in $O(nN + mN)$

DFS runs in $O(|V| + |E|) \leq O(nN + mN)$

Because of $n \leq m$ the total runtime is $O(m \cdot N)$

/ 4 P

- d) We are given two attractions $s, t \in \{1, 2, \dots, n\}$ and a starting time $T_{\text{start}} \in \{0, 1, \dots, N-1\}$. Describe an algorithm which outputs the minimum amount of time (in time-units) required for tourists to travel from s to t , starting at time T_{start} (or ∞ when no such travel is possible). Now, **tourists are willing to wait** at an attraction for a bridge to open if this reduces their total travelling time. The running time of your algorithm should be $O(mN \cdot \log(mN))$.

Hint: Use a weighted, directed graph with the same vertex set as in part b). Which edges should you add to model tourists waiting at an attraction for a bridge to open?

Official solution

1) We use Dijkstra's algorithm on a weighted, directed graph with non-negative weights.

2) We use the weighted, directed graph G whose vertex set consists of N copies of the attractions $\{1, 2, \dots, n\}$, labeled $\{(i, T) : i \in \{1, 2, \dots, n\}, T \in \{0, 1, \dots, N-1\}\}$.

We put an edge from (i, T) to (i', T') if there is a bridge $b_k = \{i, i'\} \in B$ with $T \in O_k$ and $T' = T + O_k \bmod N$. These edges have weight C_k .

For any $i \in \{1, \dots, n\}$ and $T \in \{0, \dots, N-1\}$, we also put an edge from (i, T) to $(i, T+1 \bmod N)$, called a waiting edge. These edges have weight 1, except if $i = t$, in which case they have weight 0.

3) Set $S = (s, T_{\text{start}}) \in V(G)$. We run Dijkstra's algorithm to find the shortest path from S to $(t, 0)$, and return its length. By the previous exercise, and since we added waiting edges, directed walks in G correspond to legal walks for tourists in Amsterdam. Furthermore, because the waiting edges connecting (t, T) to $(t, T+1 \bmod N)$ have weight 0, the shortest path from S to $(t, 0)$ in G has weight equal to the shortest path from S to any vertex of the form (t, T) , and this weight corresponds to the time it would take a tourist to walk that path. Our algorithm is therefore correct.

4) We run Dijkstra's algorithm on a graph with $n \cdot N$ vertices and at most $(m+n) \cdot N$ edges. Using the fact that $m \geq n$, this leads to a total running time of $O(mN \cdot \log(mN))$.

Based on negative feedback, the government decides to **open all bridges during the entire day**. In an attempt to steer traffic, they release a traveller's guide which rates how nice the view is from each bridge. As the view depends on the direction in which a bridge $b_k = \{i_k, j_k\}$ is crossed, the guide lists two numbers $R_k^+, R_k^- \in \mathbb{Z}$ (i.e. they are **possibly negative** integers). Assuming $i_k < j_k$, R_k^+ corresponds to crossing b_k from i_k to j_k , and R_k^- corresponds to crossing b_k from j_k to i_k . To avoid tourists getting lost, the government made sure that any walk around Amsterdam which starts and ends in the same attraction has total rating strictly less than 0.

/ 3 P

- d) We are given two attractions $s, t \in \{1, 2, \dots, n\}$. Describe an algorithm which outputs the **maximum** possible total rating of a walk from s to t . The running time of your algorithm should be $O(nm)$.

Hint: Use a weighted, directed graph whose vertex set consists of the attractions $\{1, 2, \dots, n\}$. (The opening times O_k and crossing times C_k do not play any role in this part of the exercise!)

1. We use the Bellman-Ford algorithm on a directed, weighted graph without negative cycles.
2. We use the weighted, directed graph G whose vertex set consists of the attractions $\{1, 2, \dots, n\}$. For each bridge $b_k = \{i_k, j_k\} \in B$ with $i_k < j_k$, we put an edge (i_k, j_k) with weight $-R_k^+$, and an edge (j_k, i_k) with weight $-R_k^-$.
3. The length of a path in G is equal to minus the rating of the corresponding walk in Amsterdam. Therefore, G has no negative cycles. We can thus use the Bellman-Ford algorithm to find a shortest path from s to t in G , which (by negation) is the walk of maximum total rating from s to t in Amsterdam.
4. We run the Bellman-Ford algorithm on a graph with n vertices and $2m$ edges. This leads to a total running time of $O(nm)$.